SYNCHRONY HACKATHON · REAL-TIME FRAUD DETECTION IN DIGITAL LENDING

Sentinel — demo runbook

Everything needed to install the project, run the backend and frontend, and record the demo. Follow it top to bottom; every command is copy-paste ready and the expected output is shown after each one.

What the demo shows

Sentinel scores a loan application in about 20 ms and returns one of four decisions, together with the reasons behind it. The recording walks through a fraud ring being caught by the system as applications arrive, then an analyst confirming fraud and the very next application from that device being blocked automatically.

DECISION	MEANING	WHEN
ALLOW	Application continues normally	risk < 0.35
STEP UP	Ask for OTP or a liveness check	risk ≥ 0.35
REVIEW	Held for a fraud analyst	risk ≥ 0.60
BLOCK	Blocked and raised as a fraud case	risk ≥ 0.85, or a hard-block rule

What you need before you start

REQUIREMENT	VERSION	CHECK WITH	NEEDED FOR
Python	3.11 or newer	`python3 --version`	Backend API and the ML models
Node.js + npm	18 or newer	`node --version`	React console
Git	any	`git --version`	Cloning the repository
A terminal	—	—	Three tabs or windows are used
A browser	Chrome / Edge / Safari	—	The console runs at localhost:5173
Disk space	about 600 MB	—	Python venv (~450 MB, mostly scikit-learn) and node_modules
Internet	first install only	—	Downloading packages. The demo itself runs fully offline

Not required

- **No cloud account, no API key.** Without LLM credentials the AI layer falls back to a deterministic explanation, which is a built-in part of the product. The console shows `rule_based_fallback` as the source so this is visible and honest on camera.
- **No database server.** SQLite is used by default and is created automatically.
- **No Docker.** Only needed for the optional PostgreSQL + pgvector path (page 6).

> **Windows users:** the commands below use `make`, which is not installed by default. Page 6 lists the equivalent commands to run directly, or use WSL / Git Bash.

PART 1

One-time setup

Roughly five minutes, most of it package downloads. You only do this once.

1 Get the code

```
git clone https://github.com/Deva2712/synchro.git
cd synchro
```

> **The fastest path is Docker.** If Docker Desktop is installed, steps 2 and 3 below are replaced by `cp .env.example .env` (fill in the three values) and `docker compose up --build`, then open **http://localhost:8000**. That needs no Python and no Node at all. Page 3 covers it. The steps below are the local development path, which gives you hot reload and visible terminals.

2 Create the configuration file

The project reads every setting from environment variables, so nothing sensitive lives in the code. Copy the template and fill in three values.

```
cp .env.example .env
python3 -c "import secrets; print('JWT_SECRET=' + secrets.token_urlsafe(32))"
```

Open `.env` and set these three lines (the JWT secret is the value printed above):

```
JWT_SECRET=<paste the generated value here>
SEED_ADMIN_PASSWORD=SentinelDemo!2026
SEED_ANALYST_PASSWORD=SentinelDemo!2026
```

Any password of 8+ characters works. These two accounts are created the first time the API starts. `.env` is git-ignored, so it is never committed.

3 Install both stacks

```
make setup
```

This creates a Python virtual environment in `.venv`, installs the backend requirements, and runs `npm install` for the console. Expected: pip and npm output ending without errors.

> **If npm skips the dev packages** (Vite fails to start later with "Could not resolve 'vite'"), your shell has `NODE_ENV=production` set. Fix it with:
>
> ```
> cd frontend && npm install --include=dev && cd ..
> ```

4 Verify the install

```
make test
```

Expected output:

```
20 passed in 1.5s
```

If all 20 pass, the decision engine, rules, PII masking, authentication and the learning loop are all working. This is also a good thing to show on camera at the start of the recording.

What gets created on first run

FILE	WHAT IT IS
`sentinel.db`	SQLite database — applications, users, fraud typologies
`sentinel_model.joblib`	Trained models (logistic regression + isolation forest) with their metrics
`sentinel_embedder.joblib`	Embedding model used for typology search

All three are git-ignored and are rebuilt automatically if deleted. Deleting `sentinel.db` is how you reset the demo to a clean state.

PART 2

Running the backend and frontend

Three terminal tabs, all opened in the project folder. Start them in this order.

Option A — everything in Docker (no Python or Node needed)

```
cp .env.example .env      # set JWT_SECRET and the two seed passwords
docker compose up --build # first build takes 3–5 minutes
```

Two containers start: `db` (PostgreSQL 16 with pgvector) and `app` (the API with the React console built inside it). Check them with:

```
docker compose ps
NAME              STATUS
synchrome-app-1   Up (healthy)
synchrome-db-1    Up (healthy)
```

Open **http://localhost:8000** — console and API on the same origin, docs at `/docs`. Push demo traffic through it:

```
docker compose exec app python -m backend.data.simulate   --n 60 --rate 5 --ring --
password "$SEED_ANALYST_PASSWORD"
```

Stop with `docker compose down`; add `-v` to wipe the data too. If only `db` appears in Docker Desktop, you are on an old version of the compose file — pull the latest code, it now contains an `app` service as well.

Option B — local development (hot reload)

Terminal 1 — the backend API

```
make api
```

Starts FastAPI on **http://127.0.0.1:8000**. On the very first start it trains the models from synthetic traffic (about 5 seconds) and loads the fraud typology knowledge base.

```
INFO  sentinel  Model ready: {'roc_auc': 0.9386, 'pr_auc': 0.8613, ...}
INFO  uvicorn   Uvicorn running on http://127.0.0.1:8000
```

Check it: open `http://127.0.0.1:8000/docs` — the interactive OpenAPI page listing every endpoint. Worth showing on camera: it proves the project is API-first and not just a UI.

Terminal 2 — the analyst console

```
make web
```

Starts Vite on **http://localhost:5173**. It proxies `/v1` calls through to the API, so the browser only ever talks to one origin and no API URL is hardcoded in the bundle.

```
VITE v5.4.21  ready in 101 ms
->  Local:   http://localhost:5173/
```

Open that address and sign in:

ROLE	EMAIL	PASSWORD	CAN DO
Analyst	`analyst@sentinel.local`	as set in `.env`	View the queue, open cases, give verdicts
Admin	`admin@sentinel.local`	as set in `.env`	Everything above, plus retraining the model

Sign in as **admin** for the recording so the "Retrain from feedback" button is visible in the top bar.

Terminal 3 — simulated application traffic

```
make demo
```

Sends 60 realistic applications through the API at about 5 per second, including a burst of six applications from one device under six different names — the fraud ring. Each line printed is a real API response:

```
APP-999EFBB86C  BLOCK    risk=0.88   18.9ms  Emulator/rooted device behind a VPN or proxy
APP-3262E868F5  ALLOW    risk=0.06   19.7ms  Loan amount vs declared income raises risk
APP-44124986C0  BLOCK    risk=0.89   17.4ms  Same device used by 5 different applicant
names in 24h

Decision mix: {'ALLOW': 50, 'STEP_UP': 3, 'REVIEW': 3, 'BLOCK': 4}
```

The console updates every two seconds, so the queue fills in live while this runs. Options you can add: `--n 100` for more applications, `--rate 2` to slow it down for the camera, `--ring` to force the fraud-ring burst (already on by default in `make demo`).

> **Ports already in use?** Change them: `.venv/bin/uvicorn backend.app.main:app --port 8001` and `cd frontend && npx vite --port 5174`. If you change the API port, also update `API_URL` for the simulator and the proxy target in `frontend/vite.config.js`.

PART 3

The demo script

About 5–6 minutes of recording. Each scene lists what to do, and what to say while doing it.

Before you hit record: reset to a clean state so the numbers on screen are tidy and the ring is the obvious story.

```
rm -f sentinel.db          # clears all previous applications
make api                   # restart terminal 1
```

1 The problem — 30 seconds

"Digital lending fraud changes every few weeks, but rule releases take months. And every genuine customer wrongly blocked is a customer lost silently. So the system has to decide in milliseconds, keep learning, and explain itself."

Show the terminal running `make test` — 20 tests passing — then move to the browser.

2 The API is the product — 30 seconds

Open `http://127.0.0.1:8000/docs`. Point out `POST /v1/applications/score`, the case endpoints, the feedback endpoint and the retrain endpoint.

"Everything is an API. The console is just one client — a loan origination system would be another."

3 Live traffic — 60 seconds

Sign in to the console as admin. Run `make demo` in terminal 3 and let the queue fill.

Point at the KPI row: *"83% straight through with no friction, and the p95 decision time is 22 milliseconds — this runs inside the application flow, not in a nightly batch."*

4 The fraud ring — 90 seconds

Click the **BLOCK** filter. Several rows will read *"Same device used by 5 different applicant names in 24h"*.

"Each of these applications looks fine on its own — no emulator, normal typing, an old account. A per-application rule engine passes all of them. What gives the ring away is the pattern across them, and we only see it because device and IP velocity are counted at the moment of scoring."

5 The case file — 90 seconds

Click one of those rows. Walk down the right-hand panel:

- **Score composition** — how the ML model, the anomaly model and the rules each contributed.
- **Evidence** — rule hits in plain English, plus the model's own per-feature contributions in log-odds.
- **Matched typology** — "Device farm / application ring", found by vector search, with the control that stops it.
- **AI analyst summary** — and the line underneath it: *generated by <source>, explanation only, the decision was made by the rules and models.*

"The language model explains, it never decides. If it is slow or down, the wording changes — the decision does not."

6 The learning loop — 60 seconds

Click **Confirm fraud**. Then in terminal 3 run `make demo` again.

New applications from that same device now come back BLOCK with the first reason *"Device or IP previously confirmed as fraud by an analyst"*.

"One analyst click protects the whole portfolio from the next application onwards — no retrain, no deployment."

7 Retraining, safely — 45 seconds

Click **Retrain from feedback** in the top bar. The banner reports the labelled case count and the ROC-AUC before and after.

"Retraining a live credit control is risky, so a new model is only promoted if it is not meaningfully worse than the current one. Here it was within noise, so the existing model kept serving — and the API says so."

PART 4

Recording and submitting

Recording checklist

- Record at **1920×1080** or the full screen. Zoom the browser to 100–110% so text is readable after compression.
- Arrange the window so the console fills most of the screen, with one terminal visible for the simulator output.
- Keep it to **5–7 minutes**. Speak over it — a silent screen recording is much weaker.
- Tools: QuickTime (macOS, `File → New Screen Recording`), Xbox Game Bar (Windows, `Win+G`), or OBS on any platform.
- Close anything personal — notifications, other tabs, bookmarks bar.

Files to submit

FILE	WHAT IT IS
`<rollnumber>.pdf`	The 18-slide solution deck (`deck/Sentinel.pdf`, renamed)
`<rollnumber>.mp4`	The screen recording
`<rollnumber>.zip`	Both of the above, zipped together

Put your name and roll number on the deck and rebuild it:

```
.venv/bin/python deck/build.py --pdf --author "Your Name" --roll "21XX1234"
cp deck/Sentinel.pdf ~/Desktop/21XX1234.pdf
```

Then zip and email:

```
cd ~/Desktop && zip 21XX1234.zip 21XX1234.pdf 21XX1234.mp4
```

> **Deadline:** email to `Technologyinterns@syf.com` before **12:00 PM IST on 21 August**. Anything later is not accepted. Use only the roll number in the file names, as the instructions require.

Questions you should be ready for

QUESTION	SHORT ANSWER
Why logistic regression and not something bigger?	Because it is linear, every feature's contribution is exact arithmetic (`coefficient × scaled value`), so the explanation is the decision itself and not an approximation. In a regulated credit process that matters more than a few points of AUC.
Why an isolation forest as well?	It uses no labels at all, so it still fires on fraud patterns nobody has labelled yet — the "new fraud vector" part of the problem statement.
Where does the LLM sit?	After the decision, off the customer path. It explains; it cannot change or delay a decision.
Is the data real?	No — it is synthetic, and deliberately hard: 40% of fraud cases show only some warning signs, 1.5% of labels are wrong, and genuine users share devices and use VPNs. An early version scored AUC 1.00, which just meant the fake fraud was too obvious.
How would this scale?	The API is stateless, so it scales horizontally. The two changes needed are Redis-based rate limiting and pgvector for typology search — both are listed in the README.
What about false positives?	That is why there are four outcomes instead of two. Suspicion buys an OTP; blocking needs strong evidence. The false-positive rate measured on held-out data is 3.9%.

PART 5

Optional extras and troubleshooting

Running without make (Windows or no make installed)

INSTEAD OF	RUN
`make setup`	`python -m venv .venv` then `.venv\Scripts\pip install -r backend/requirements.txt` then `cd frontend && npm install --include=dev`
`make api`	`.venv\Scripts\uvicorn backend.app.main:app --reload --port 8000`
`make web`	`cd frontend && npm run dev`
`make demo`	`.venv\Scripts\python -m backend.data.simulate --n 60 --rate 5 --ring`
`make test`	`.venv\Scripts\python -m pytest backend/tests -q`

On macOS and Linux replace `.venv\Scripts\` with `.venv/bin/`.

Optional: PostgreSQL with pgvector

The same code runs on PostgreSQL without any changes — only the connection string differs.

```
docker compose up -d
export DATABASE_URL=postgresql+psycopg://sentinel:sentinel@localhost:5432/sentinel
.venv/bin/pip install "psycopg[binary]"
make api
```

Optional: a live LLM instead of the fallback

Add one of these to `.env` and restart the API. The console top bar will change from "LLM offline" to the model name, and case summaries will be written by the model.

```
# Option A - Anthropic API
ANTHROPIC_API_KEY=sk-ant-...
LLM_BACKEND=anthropic

# Option B - the same model on AWS Bedrock (uses your IAM role or profile)
LLM_BACKEND=bedrock
AWS_REGION=ap-south-1
```

Either way the key is read from the environment only. Nothing is stored in the code and `.env` is git-ignored.

Troubleshooting

SYMPTOM	CAUSE AND FIX
Vite fails: *Could not resolve 'vite'*	`NODE_ENV=production` makes npm skip dev packages. Run `cd frontend && npm install --include=dev`.
Console shows "No applications yet"	The database is empty — run `make demo` in a third terminal.
Login fails with "Invalid credentials"	The passwords in `.env` were set *after* the first API start, so the accounts already exist with the old ones. Delete `sentinel.db` and restart the API.
No "Retrain from feedback" button	You are signed in as the analyst. Retraining is admin-only — sign in as `admin@sentinel.local`.
Retrain returns 403	Same reason as above — the endpoint requires the admin role.
"Session expired — sign in again"	Tokens last 60 minutes, and restarting the API invalidates them if no `JWT_SECRET` is set in `.env`. Sign in again.
Port 8000 or 5173 already in use	Run on another port (see page 3), or stop the other process: `lsof -ti:8000 | xargs kill`.
Docker shows only the `db` container	You have an older `docker-compose.yml`. Pull the latest code — it defines an `app` service too — then `docker compose up --build`.
"This site can't be reached" on localhost	Nothing is serving that port. With Docker use **:8000** (not :5173) and check `docker compose ps` shows `app` healthy; locally make sure both `make api` and `make web` are running.
App container exits on first start	Check `docker compose logs app`. Usually the database was not ready — `docker compose up` again; the app waits for the db healthcheck.
Everything looks stale	Reset completely: `rm -f sentinel.db *.joblib` then restart the API. The models retrain automatically in about five seconds.

REFERENCE

Command and API reference

Every command in one place

COMMAND	WHAT IT DOES
`make setup`	Creates the venv, installs backend and frontend packages
`make api`	Runs the API on port 8000 (docs at `/docs`)
`make web`	Runs the analyst console on port 5173
`make demo`	Sends 60 simulated applications, including a fraud ring
`make test`	Runs the 20 tests
`make docker`	Builds and runs everything in containers on port 8000
`make docker-demo`	Pushes simulated traffic through the containerised API
`make docker-down`	Stops the containers
`python deck/build.py --pdf`	Rebuilds the solution deck PDF
`python deck/guide.py --pdf`	Rebuilds this runbook

API endpoints

METHOD	ENDPOINT	PURPOSE
POST	`/v1/auth/login`	Get a JWT (valid 60 minutes)
GET	`/v1/me`	Current user and role
POST	`/v1/applications/score`	**Score one application in real time**
GET	`/v1/applications`	Case queue, filterable by decision
GET	`/v1/applications/{id}`	Full case: features, evidence, typologies, summary
POST	`/v1/applications/{id}/feedback`	Analyst verdict (fraud / legitimate)
GET	`/v1/model/info`	Model metrics, weights and thresholds in use
POST	`/v1/model/retrain`	Retrain from feedback (admin only)
GET	`/v1/metrics`	Operational metrics for the dashboard
GET	`/health`	Liveness check

Scoring one application from the terminal

Useful if you want to show the raw API response on camera instead of the UI.

```
TOKEN=$(curl -s -X POST localhost:8000/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"analyst@sentinel.local","password":"SentinelDemo!2026"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

curl -s -X POST localhost:8000/v1/applications/score \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d @backend/data/sample_application.json | python3 -m json.tool
```

The response contains the decision, the risk score, the three component scores, the ranked evidence and the matched typologies — everything the console displays.

Repository

`https://github.com/Deva2712/synchro`

README.md has the architecture diagram, the full API reference, the security notes and the list of known limitations.